

Entity Generation CLI - User Guide

Part 1 — Backend Setup

Step 1 — System Requirements

Ensure your system has the following installed:

Item	Required
Docker or Podman	✓ Yes
Java 17+	✓ Yes
curl	✓ Yes
Linux/macOS Shell	✓ Yes

Step 2 — Launch Backend Service

1. Open a terminal.
2. Navigate to the backend service folder:

```
cd backend-service
```

3. Make the script executable (first time only):

```
chmod +x start.sh
```

4. Run the script:

[./start.sh](#)

5. Select runtime:

- 1 → Docker
- 2 → Podman

The script will automatically:

- Load available `.tar` image
- Create network `cd-network`
- Start backend service container

Step 3 — Validate Service is Running

Check that the backend service container is running:

- Docker:

`docker ps`

- Podman:

`podman ps`

Expected Output:

Container Name	Status
connector_designer_services	Running

⚠ If containers are not running, check logs:

```
backend-service/logs/cd/
```

Part 2 — Entity Generation & Packaging CLI

Step 1 — Prerequisites

Before running the script, ensure the following:

Item	Required
Java 17+	✓ Yes
Backend Service Running	✓ Yes (local or Docker)
Bash Shell (Linux/macOS)	✓ Yes
curl	✓ Yes

The backend service should be running at: <http://localhost:6002>

Step 2 — Script Setup

1. Open a terminal.
2. Navigate to the CLI folder:

```
cd cli
```

3. Make the script executable (first time only):

```
chmod +x generate-entities.sh
```

4. Run the script:

[`./generate-entities.sh`](#)

Step 3 — Provide Database Details

The script will prompt for the following:

Prompt	Example
JDBC Driver Class	org.postgresql.Driver
JDBC URL	jdbc:postgresql://localhost:5435/CONNECTOR_DESIGNER_TEST
DB Username	postgres
DB Password	postgres
Schema Name	public
Hibernate Dialect	org.hibernate.dialect.PostgreSQLDialect
Output Package	com.solace.test
Output Directory (absolute path)	/Users/example/output

The script automatically updates:

- `hibernate.properties`
 - `reveng.xml`
-

Step 4 — Run Entity Generator

The script executes the generator JAR:

```
java -jar generate-entities-cli-1.0-SNAPSHOT.jar \  
-c hibernate.properties \  

```

```
-r reveng.xml \  
-o /output/path \  
-p com.solace.test
```

✓ Generates entity classes for all tables in the schema.

Step 5 — CURL Request for Backend Integration

Provide table-specific details:

Prompt	Example
Table Name	customer
Strategy	1 → readingIndicator 2 → timestamp 3 → sequential
Child Tables (optional)	customer_address, customer_order
Auto-populate NOT NULL columns	true / false
CUSTOM_CONFIGURATION (if false)	TABLE.COL,...
Strategy Indicator	read_column_xyz

2. Select backend service mode:

- **1** → Local → uses local output paths
- **2** → Docker → sets paths inside container automatically

3. The script generates a JSON payload and prints a curl command:

```
curl --location --request POST  
'http://localhost:6002/cd/cli/update/entity/files' \  
--header 'Content-Type: application/json' \  
--data-raw '{...}'
```

4. Choose whether to execute the curl now (y/n).

✓ This updates the backend service with generated entity files.

Step 6 — Package Updated Entity Files

1. Confirm the source folder for updated entities:

Default: `/output/path/updated_entity/com/solace/test`

2. Specify the output JAR path (absolute path with filename):

Example: `/Users/example/output/updated-entity.jar`

3. The script runs the packager JAR:

```
java -jar package-jar-cli-1.0-SNAPSHOT.jar -s /source/path -o  
/output/path/updated-entity.jar
```

✓ The generated JAR contains all updated entity files.

Step 7 — Output Directory Structure

After running, the output directory should look like:

```
output/  
├─ entity/  
├─ updated_entity/  
└─ updated-entity.jar
```

Tips & Notes

- Always use absolute paths for output folders and JAR files.
- If running in Docker mode, paths inside the container are automatically set.
- Use `curl` only after the backend service is confirmed to be running.
- Auto-populate `true` allows NOT NULL columns to be automatically handled.
- Custom configuration is required only if auto-populate is `false`.
- If running in podman mode, give permissions to the log folder e.g, `sudo chmod -R 777 backend-service/logs/cd`